# Bases de datos orientadas a objetos, pruebas y documentación

*3. BD orientadas a objetos, pruebas y documentación*

# BD orientadas a objetos, pruebas y documentación

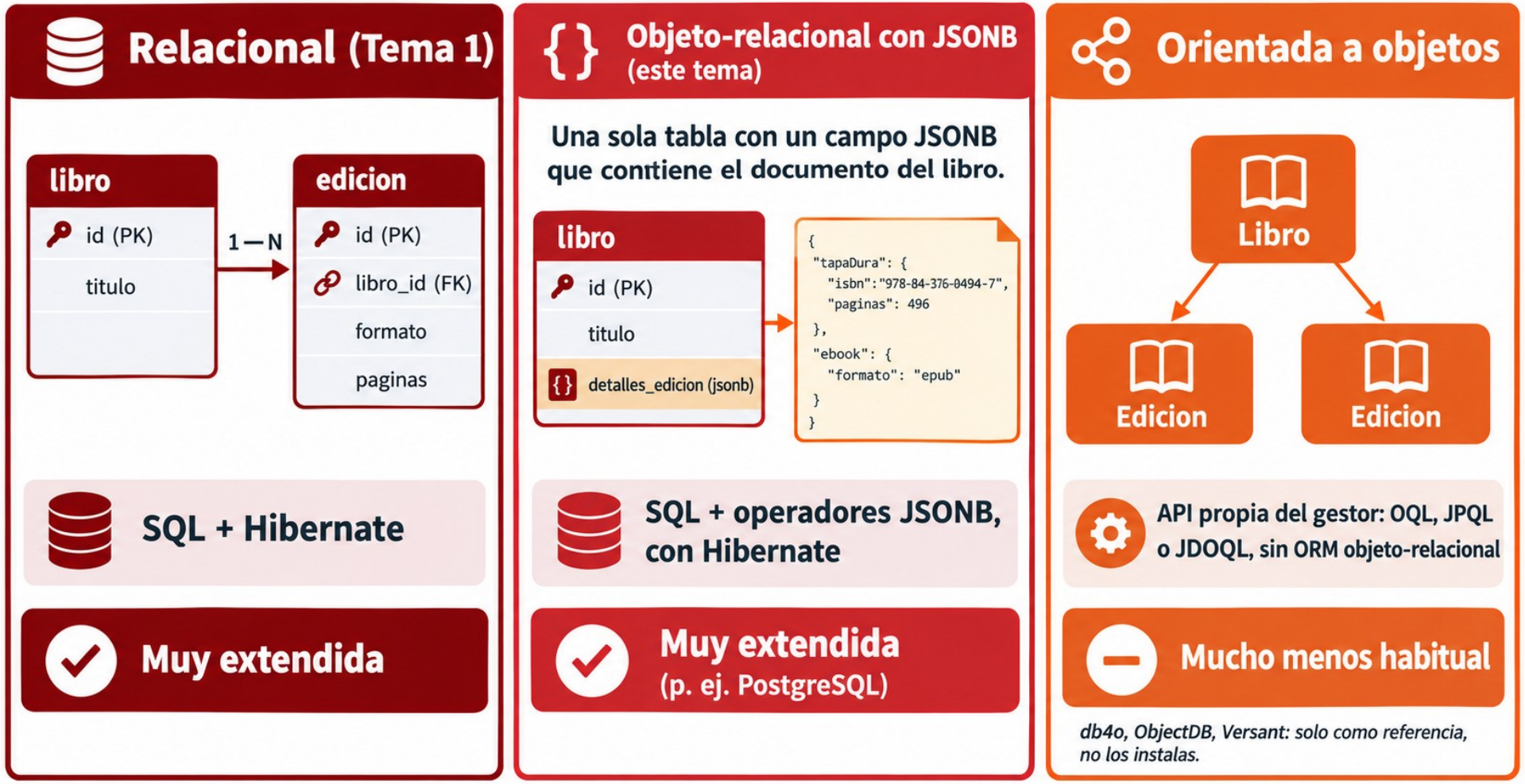
Este apartado tiene dos partes bien diferenciadas: primero un bloque teórico sobre una categoría de bases de datos que no vas a usar en las prácticas —lo ves de forma conceptual, sin instalar ni configurar nada—, y después las pruebas que dan por buenas todas las piezas construidas hasta ahora.

# Bases de datos orientadas a objetos puras

Una base de datos orientada a objetos almacena directamente el estado de los objetos, su identidad y las relaciones que mantienen entre ellos, sin descomponerlos previamente en el modelo relacional de filas, columnas y claves foráneas.

Esto no significa que copie literalmente un objeto Java tal como existe en la memoria del programa. El gestor utiliza su propia representación interna, pero su modelo lógico se basa en objetos y referencias, no en tablas relacionales.

# Los tres modelos, uno junto a otro



# Desventaja, ORM, OQL y gestores de referencia

## La desventaja real

Menos documentación, menos gente con experiencia a la que preguntar y el riesgo de quedarte atado a un gestor que puede dejar de mantenerse. Por eso casi nadie la elige hoy para un proyecto nuevo.

## Sin ORM objeto-relacional

Al no existir la separación entre objetos y tablas, no hace falta un ORM como Hibernate. Sigue haciendo falta una API o un controlador para comunicarse con el gestor, pero ya no traduce entidades a filas y claves foráneas.

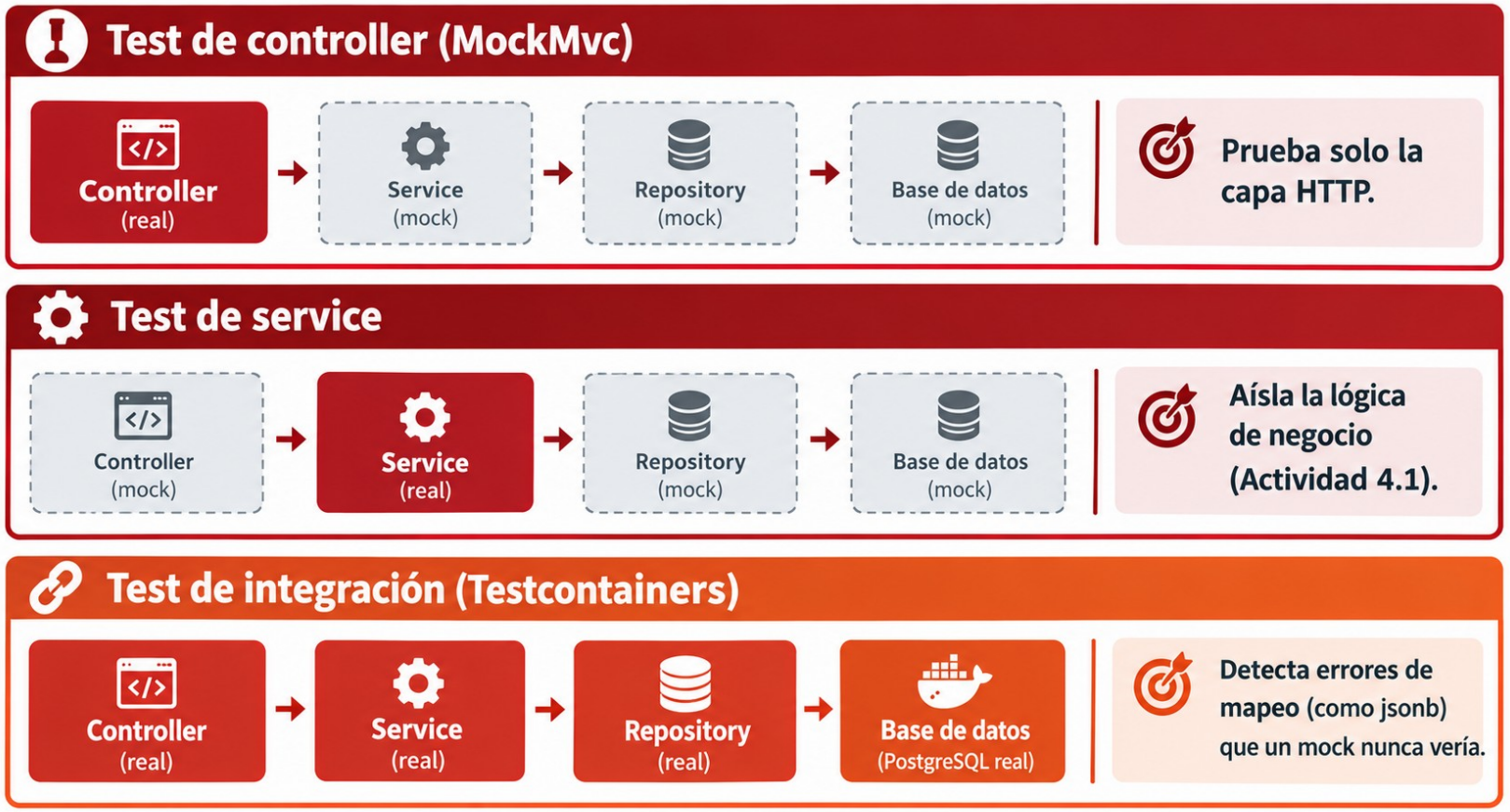
## OQL, uno entre varios

OQL (Object Query Language) está asociado históricamente a este modelo, pero no es universal: otros gestores, como ObjectDB, consultan con JPQL o JDOQL sin traducir después a SQL.

## Solo como referencia

db4o, ObjectDB y Versant son ejemplos representativos, algunos principalmente históricos. No todos usan el mismo lenguaje de consulta ni las mismas APIs. No vas a instalar ninguno: este bloque es conceptual.

# Pruebas y documentación: tres tipos de test




Testcontainers levanta contenedores Docker reales (una base de datos, una cola de mensajes...) solo durante el test y los destruye al terminar, sin dejar nada instalado. Aparece por primera vez en el curso.

# Por qué se fija Testcontainers 1.20.4

## Una versión fijada a propósito

spring-boot-starter-parent gestiona también las versiones de muchas dependencias externas, entre ellas Testcontainers. Aun así, aquí se fija la 1.20.4, porque los módulos y los imports de las actividades corresponden a la línea 1.x.

Testcontainers 2 cambió los nombres de sus módulos y movió clases a paquetes nuevos: usar sin más la versión de Spring Boot obligaría a adaptar coordenadas Maven e imports.

## Qué es un BOM

Un BOM (Bill of Materials, "lista de materiales") es un POM especial que coordina las versiones de un conjunto de artefactos compatibles, pero no añade por sí mismo esas librerías al proyecto.

Al importar el BOM de Testcontainers 1.20.4, el proyecto fija la versión de todos sus módulos y mantiene las coordenadas e imports de las actividades.

# El BOM va junto al <parent>

```
<properties>
    <testcontainers.version>1.20.4</testcontainers.version>
</properties>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>testcontainers-bom</artifactId>
            <version>${testcontainers.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

# Con el BOM importado, sin <version> individual

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-testcontainers</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <scope>test</scope>
</dependency>
```

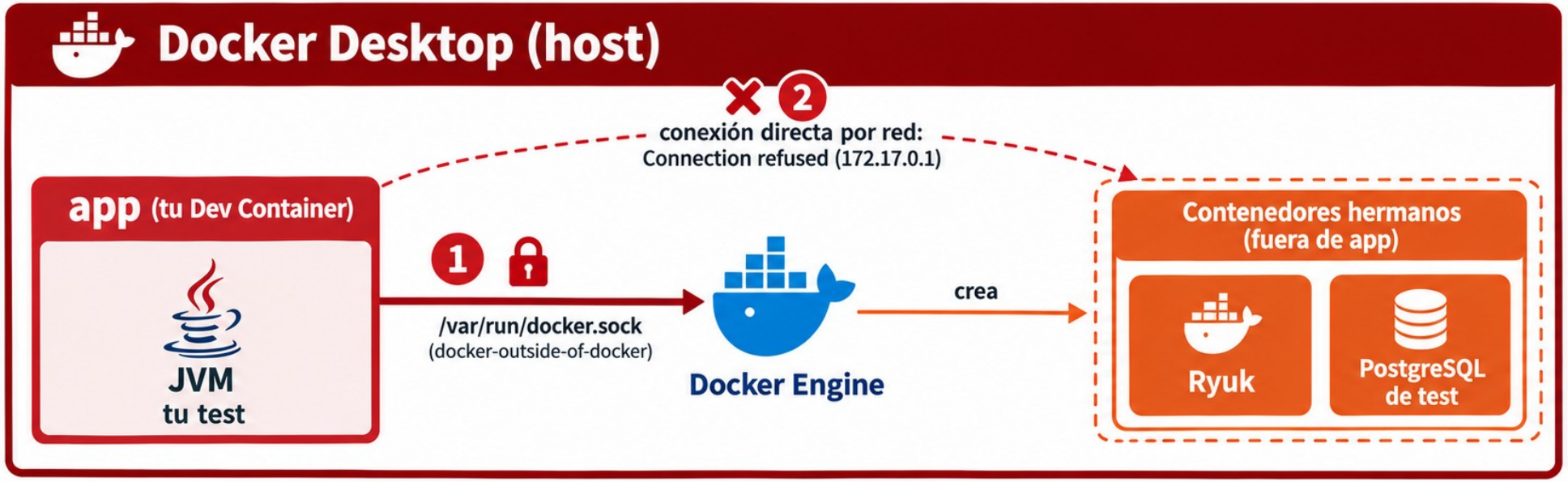
Esto deja el terreno preparado para el resto del curso: cuando más adelante se añadan otros contenedores (MongoDB, RabbitMQ...), ninguno necesitará tampoco su propia <version> — todos quedan cubiertos por este mismo BOM.

# Por qué esto funciona dentro del Dev Container

Testcontainers necesita arrancar contenedores de verdad — y el proyecto entero corre dentro de un contenedor (app) desde la Actividad 1.1. Funciona porque, ya desde entonces, se ha montado el socket de Docker del host dentro de app y se ha añadido la feature docker-outside-of-docker: el contenedor puede pedirle contenedores nuevos al mismo Docker que usa el sistema operativo, sin necesitar un Docker propio dentro del Docker.

Esa misma idea —la JVM, dentro de app, pidiéndole contenedores al Docker del host— es justo lo que puede fallar de dos maneras distintas, cada una en un momento distinto del proceso.

# Los dos fallos posibles, de un vistazo




**1 — Permission denied: no puede ni hablar con Docker**

```
"postStartCommand": "sudo chmod 666
/var/run/docker.sock || true"
```

en devcontainer.json

**2 — Connection refused: habla con Docker, pero no alcanza lo que Docker crea.**

```
TESTCONTAINERS_HOST_OVERRIDE=host.docker.internal
```

en docker-compose.yml, servicio app

# Permission denied y Connection refused: causa y solución

## 1. Permission denied

En Docker Desktop el socket /var/run/docker.sock pertenece al grupo root, y la feature no puede añadir al usuario del contenedor a root por seguridad: se queda sin permiso, antes de crear ningún contenedor.

En devcontainer.json (no en docker-compose.yml):

```
"postStartCommand": "sudo chmod 666
/var/run/docker.sock || true"
```

El || true evita que el arranque falle. Después, reconstruir el contenedor.

## 2. Connection refused

Ryuk (el contenedor auxiliar que limpia al terminar) y el PostgreSQL de test arrancan como hermanos en la red del host, no en la de app. El síntoma: Connection refused hacia una IP tipo 172.17.0.1.

En .devcontainer/docker-compose.yml, servicio app:

```
environment:
  - TESTCONTAINERS_HOST_OVERRIDE=host.docker.internal
```

host.docker.internal es el nombre que Docker Desktop resuelve bien desde cualquier contenedor. Después, reconstruir el contenedor.

# Con las dependencias resueltas: la clase de test

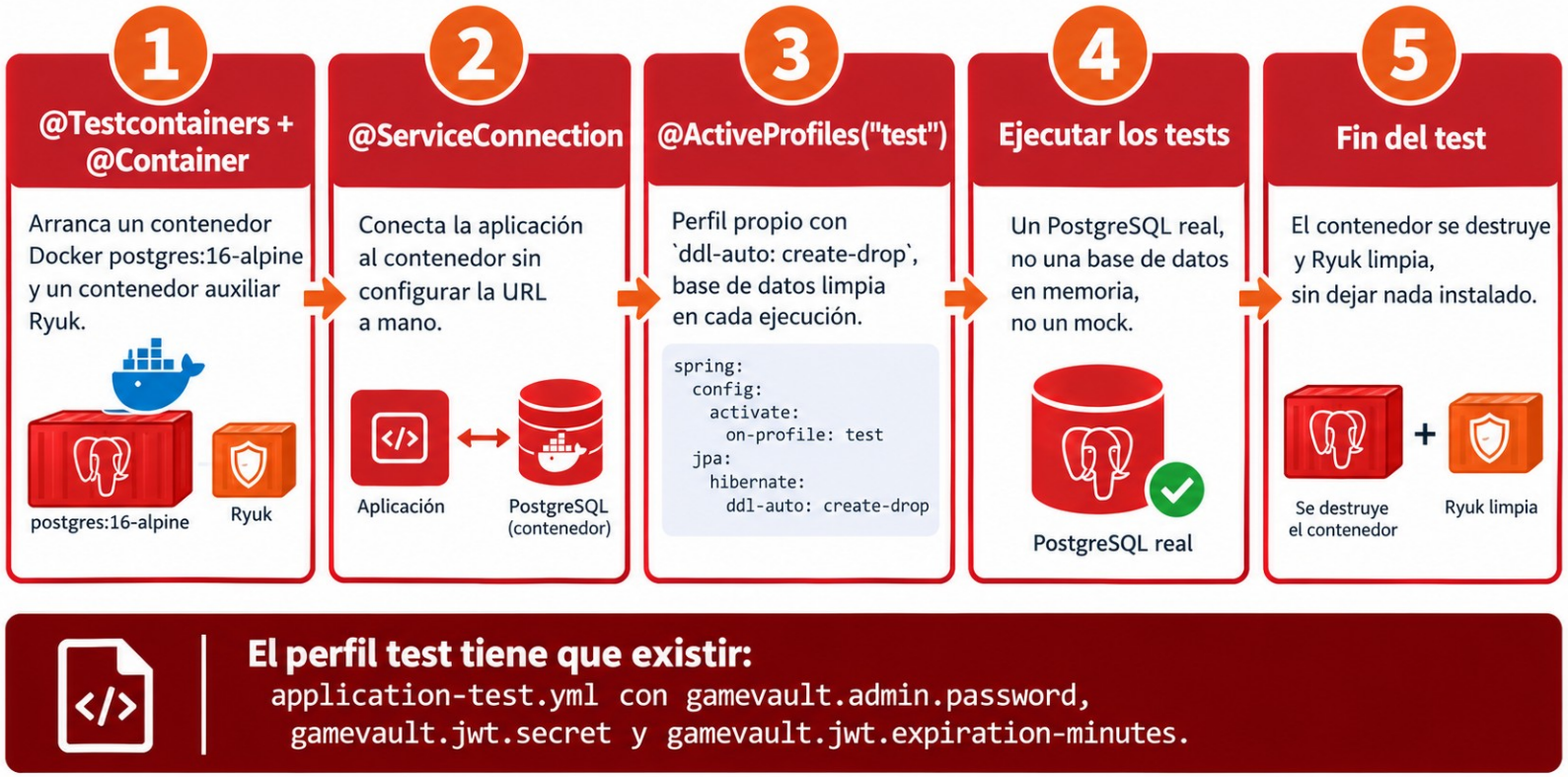
```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
@Testcontainers
class LibreriaApiTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres =
            new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired
    private MockMvc mockMvc;

    // ...
}
```

# Qué hace cada anotación: el ciclo de un test




Prueba el mapeo JSONB real contra un PostgreSQL real: una base en memoria genérica (como H2) podría no soportar jsonb ni jsonb_exists, y un mock nunca ejecuta SQL de verdad contra ningún motor.

# La anotación sola no basta: el perfil test debe existir

@ActiveProfiles("test") le dice a Spring qué perfil activar, pero no crea nada por sí sola. Tres propiedades sin valor impiden arrancar del todo: la contraseña del admin, el secreto JWT y la expiración del token — ninguna lleva valor por defecto. spring.datasource no hace falta escribirlo: eso ya lo resuelve @ServiceConnection. Crear src/test/resources/application-test.yml:

```
spring:
  config:
    activate:
      on-profile: test
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: true

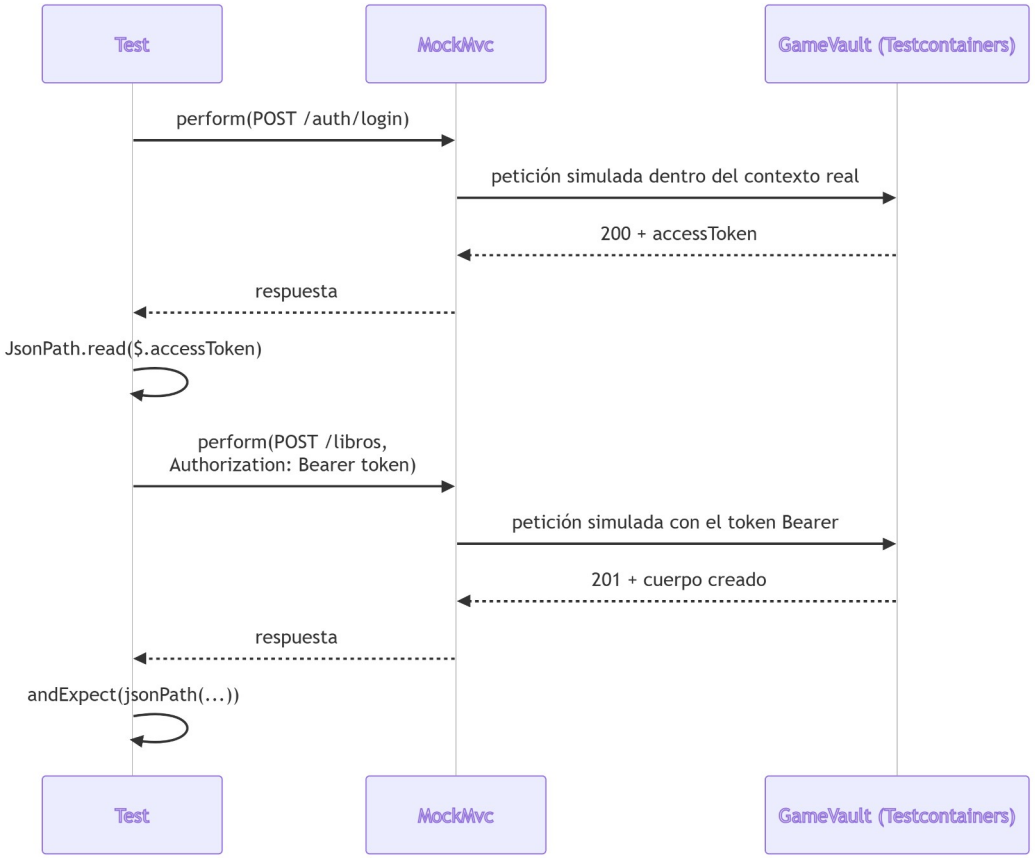
gamevault:
  admin:
    password: admintest123
  jwt:
    secret: un-secreto-de-test-de-al-menos-32-caracteres
    expiration-minutes: 60
```

# Peticiones autenticadas dentro del test

Si la API tiene rutas protegidas —y a estas alturas del curso ya las tiene, con roles y todo—, el test debe recorrer el mismo flujo de autenticación y autorización de la aplicación: no hay ningún atajo que salte la seguridad "porque es un test".

MockMvc no abre una conexión HTTP real ni arranca un servidor. Construye peticiones y respuestas simuladas, pero las hace pasar por el DispatcherServlet, la cadena de filtros de seguridad, los controllers y el resto del contexto real de Spring. Por eso se puede hacer login, extraer el token y utilizarlo después en otra petición.

# Login, token y petición autenticada




MockMvc hace pasar la petición por el contexto real de Spring — login, extracción del token con JsonPath, y una segunda petición autenticada con ese token.

# El ayudante que hace login y devuelve el token

```
private String loginComoAdmin() throws Exception {
    String respuesta = mockMvc.perform(post("/api/v1/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"username":"admin","password":"admintest123"}
                """))
        .andExpect(status().isOk())
        .andReturn().getResponse().getContentAsString();

    return JsonPath.read(respuesta, "$.accessToken");
}
```

loginComoAdmin() no lleva @Test: es un método ayudante que cualquier otro test de la clase puede utilizar. Requiere import com.jayway.jsonpath.JsonPath, ya incluido en spring-boot-starter-test.

# El test que usa el token: crearLibro

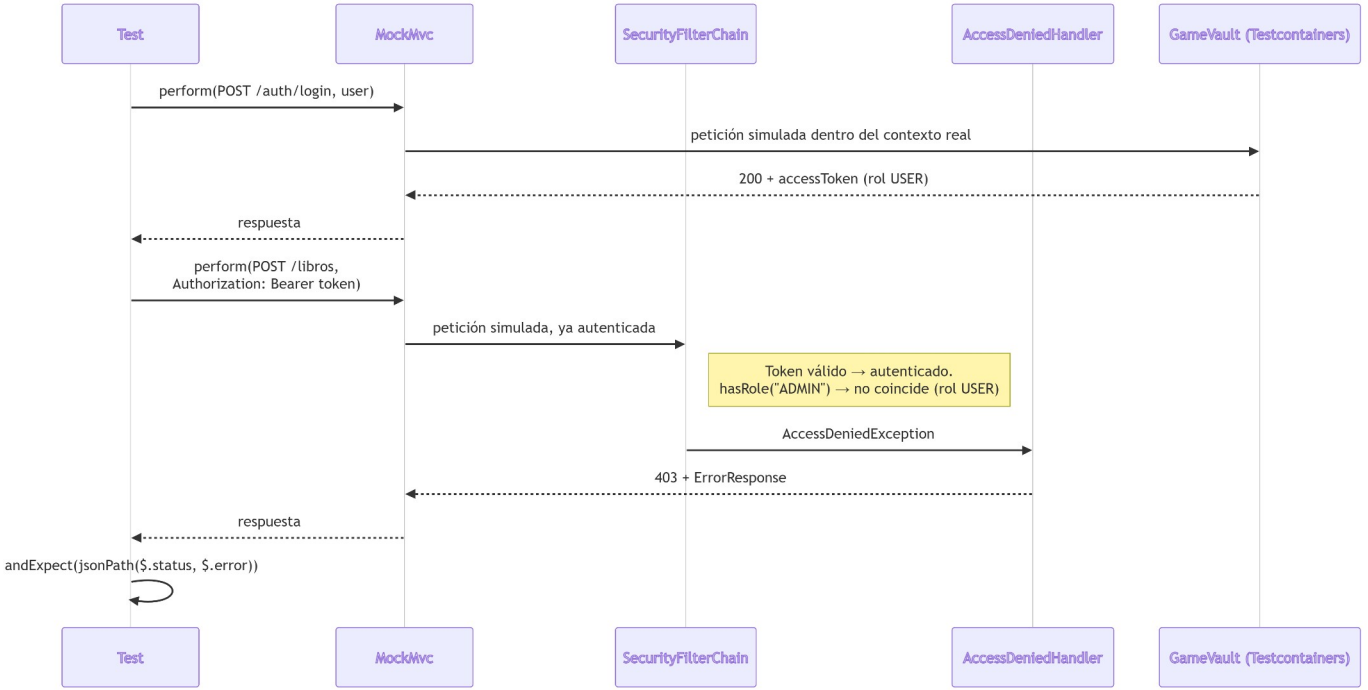
```
@Test
void crearLibro_DebeGuardarDetallesEdicion_CuandoEsValido() throws Exception {
    String adminToken = loginComoAdmin();

    mockMvc.perform(post("/api/v1/libros")
            .header("Authorization", "Bearer " + adminToken)
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"titulo":"Rayuela","detallesEdicion":
                 {"editorial":"Sudamericana","paginas":634}}
                """))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.titulo").value("Rayuela"))
        .andExpect(jsonPath("$.detallesEdicion.editorial").value("Sudamericana"));
}
```

El test ejecuta el flujo completo de creación contra un PostgreSQL real y comprueba tanto el código de estado como el cuerpo, incluida la propiedad anidada detallesEdicion.editorial.

# Comprobar también los casos de error

Un test de integración que solo prueba el camino feliz deja fuera la mitad del comportamiento real: qué pasa cuando la petición no debería tener éxito. Aquí la petición está autenticada —el token de user es válido—, pero no autorizada, así que ni siquiera llega al controller:



# El test que reproduce el flujo 403

```
@Test
void crearLibro_DebeDevolver403_CuandoElUsuarioNoEsAdmin() throws Exception {
    String userToken = loginComoUsuario(); // igual que loginComoAdmin(), sin rol ADMIN

    mockMvc.perform(post("/api/v1/libros")
            .header("Authorization", "Bearer " + userToken)
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"titulo":"Rayuela","detallesEdicion":
                 {"editorial":"Sudamericana","paginas":634}}
                """))
        .andExpect(status().isForbidden())
        .andExpect(jsonPath("$.status").value(403))
        .andExpect(jsonPath("$.error").value("No autorizado"));
}
```

No es casualidad que este test compruebe el cuerpo del error, no solo el código 403: dos errores distintos pueden compartir el mismo código de estado.

# Por qué el cuerpo del error, no solo el status

Solo el cuerpo —con el ErrorResponse propio, no el genérico de Spring— dice cuál de los dos errores ha ocurrido de verdad. Quedarse solo con status().isForbidden() dejaría pasar una regresión real: por ejemplo, que el 403 siga llegando pero con el formato por defecto de Spring Security, porque alguien ha desconectado sin querer el AccessDeniedHandler.

Qué comprobar en un test de integración, casi siempre: el código de estado, la forma del cuerpo cuando lo hay (jsonPath sobre los campos que de verdad importan), y —cuando la petición debía cambiar algo— que ese cambio ha persistido de verdad, con una segunda petición que lo confirme.

# Ideas clave

1. Una BD orientada a objetos almacena estado, identidad y referencias sin descomponer en tablas; no necesita un ORM objeto-relacional, aunque sí una API para comunicarse con el gestor.

2. OQL es uno de los lenguajes asociados históricamente a este modelo, pero no es universal: cada gestor puede usar OQL, JPQL, JDOQL o APIs propias.

3. Comparado con relacional y objeto-relacional (JSONB), lo orientado a objetos puro tiene poquísima adopción: por eso este bloque es conceptual.

4. Test de controller: capa HTTP con el service mockeado. Test de service: repositorio mockeado (Actividad 4.1). Test de integración: motor real en Docker con Testcontainers.

# Ideas clave (cont.)

1. Testcontainers detecta errores de mapeo real (como con jsonb) que un mock nunca podría detectar.

2. Contra rutas protegidas, MockMvc reproduce el flujo real de login, filtros y autorización y extrae el token con JsonPath, aunque use peticiones simuladas y no un servidor HTTP real.

3. Los casos de error también se comprueban con jsonPath, campo a campo del ErrorResponse: no basta con el código de estado, porque dos errores distintos pueden compartirlo.

4. Un test bien nombrado es documentación ejecutable: si cambia el comportamiento que comprueba, falla. No sustituye toda la documentación y solo cubre los casos que prueba.